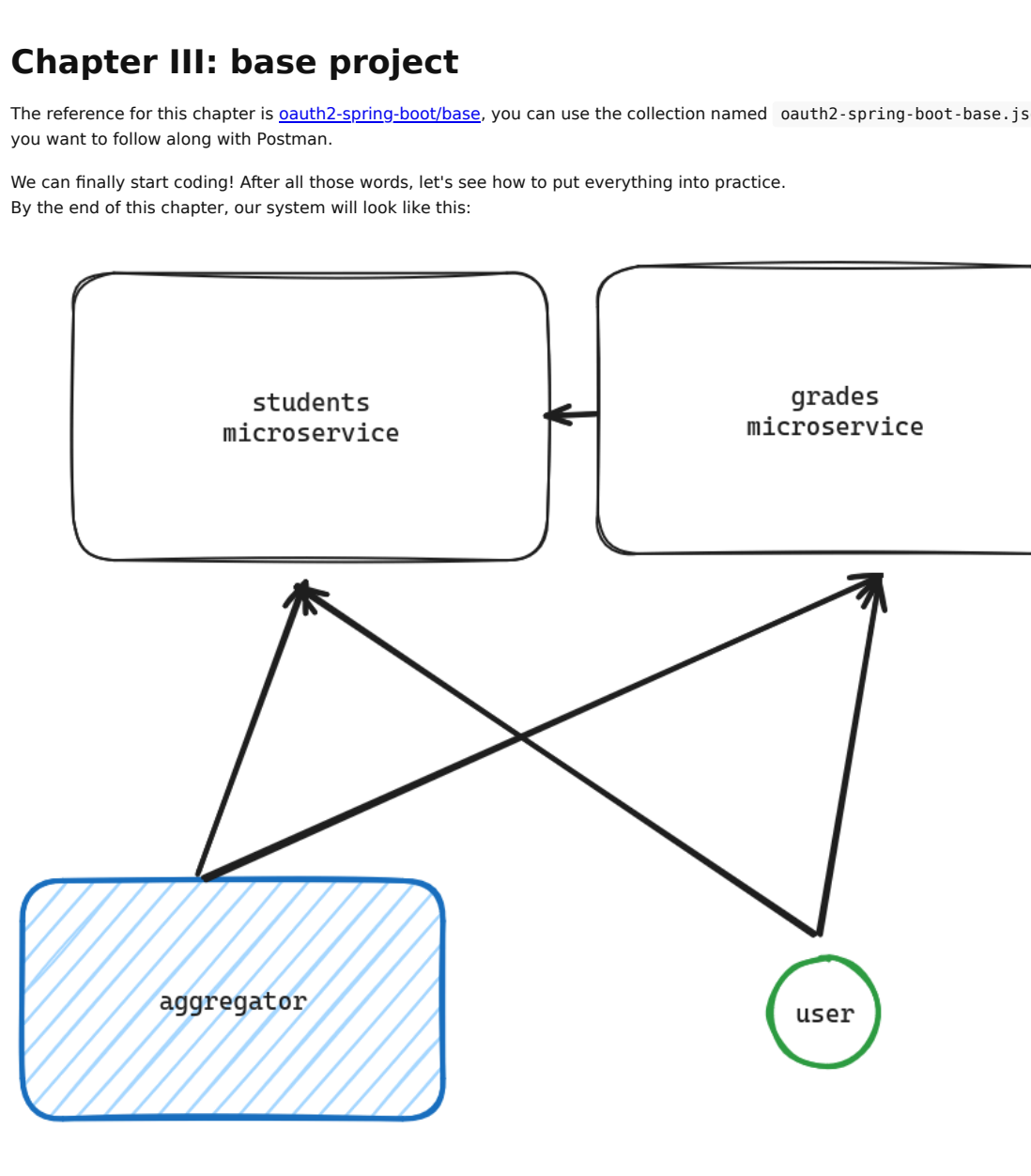
Table of contents

- Chapter III: base project
  - Booting up
  - The students microservice
  - The grades microservice
  - The aggregator
  - Checking that everything works

Chapter III: base project

The reference for this chapter is oauth2-spring-boot/base, you can use the collection named `oauth2-spring-boot-base.json` if you want to follow along with Postman.

We can finally start coding! After all those words, let's see how to put everything into practice.
By the end of this chapter, our system will look like this:



We will build our basic blocks and then extend all the other functionalities from there. Notice how there is no gateway nor authentication in place: we will add it in Chapter IV.
The project structure is pretty recognizable: it's a monorepo with multiple gradle modules (our microservices), this way we can quickly debug and develop our simple architecture.

```
.
├── build.gradle.kts
├── compose.yml
├── gradle
├── gradlew
├── gradlew.bat
├── microservices
│   ├── aggregator
│   ├── grades
│   └── students
├── README.md
├── schema
│   ├── grades
│   │   └── schema.sql
│   └── students
│       └── schema.sql
└── settings.gradle.kts
```

There's also a `schema` folder that contains database schema files for our PostgreSQL instances that we will use as storage for our microservices.
As previously mentioned, everything will be ran through Docker, hence the `compose.yml` file. Let's take a look:

```yaml
networks:
  microservices-net:
    name: microservices-net

services:
  aggregator:
    depends_on:
      - students
      - grades
    networks:
      - microservices-net
    build:
      context: microservices/aggregator
      dockerfile: Dockerfile
    mem_limit: 512m
    environment:
      - STUDENTS_URI=http://students:8080
      - GRADES_URI=http://grades:8080

  students:
    depends_on:
      students_pg:
        condition: service_healthy
    networks:
      - microservices-net
    build:
      context: microservices/students
      dockerfile: Dockerfile
    mem_limit: 512m
    environment:
      - SPRING_PROFILES_ACTIVE=docker
      - DB_CONNECTION_STRING=postgresql://test:test@students_pg:5432/students
    ports:
      - "7776:8080"

  grades:
    depends_on:
      grades_pg:
        condition: service_healthy
    networks:
      - microservices-net
    build:
      context: microservices/grades
      dockerfile: Dockerfile
    mem_limit: 512m
    environment:
      - SPRING_PROFILES_ACTIVE=docker
      - DB_CONNECTION_STRING=postgresql://test:test@students_pg:5432/grades
    ports:
      - "7777:8080"

  students_pg:
    image: postgres
    environment:
      - POSTGRES_DB=students
      - POSTGRES_USER=test
      - POSTGRES_PASSWORD=test
    healthcheck:
      test: [ "CMD-SHELL", "pg_isready -d students -U test" ]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - microservices-net

  grades_pg:
    image: postgres
    environment:
      - POSTGRES_DB=grades
      - POSTGRES_USER=test
      - POSTGRES_PASSWORD=test
    healthcheck:
      test: [ "CMD-SHELL", "pg_isready -d grades -U test" ]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - microservices-net
```

As you can probably already tell by the configuration, services are configured pretty similarly:

- the `aggregator` will wait for `students` and `grades` to start up;
- the `students` service, representing the students microservice, will connect to the `students_pg` database through the specified connection string. It will also *wait* for it to come online via the `depends_on` property. This service will be mapped on the port `7776` of the host;
- the `grades` service is configured the same way, but is mapped to another database, `grades_pg`, and to another local port, `7777`;
- the two database services, `students_pg` and `grades_pg`, are configured with easy passwords for development purposes. They are also equipped with an `healthcheck` mechanism, that will signal when the Postgres instance will be healthy and able to recieve connections (more information about this here);
- everything is connected using a Docker network: this will isolate our stack from others.

Booting up

To boot this scenario up, navigate to the branch and follow this steps:

1. `cd app`
2. `./gradlew build`
3. `docker compose up --build`
4. Wait until every service has come up
5. `cd schema`
6. `docker exec -i app-students_pg-1 psql -U test students < students/schema.sql` (this imports the database)
7. `docker exec -i app-grades_pg-1 psql -U test grades < grades/schema.sql`

The students microservice

The first microservice we are going to take a look at is the students' one.
This microservice represents a student inside a fictional school reporting system: each student will have their personal details and one or more grades.
The student microservice will only have direct access to student's personal details.
This microservice exposes three mappings:

- `GET /{studentId}` : gets a student's details;
- `POST /` : inserts a student;
- `GET /` : gets all students.

The unit is pretty simple: it uses a repository, `StudentsRepository` for accessing the database, which has the simplest schema possible:

```sql
create table students (
    id serial primary key,
    name varchar(100),
    surname varchar(100)
);
```

You can check all the details here and inside the Postman collection.

The grades microservice

This microservice will represent the grades assigned to a certain student. The grades microservice will only have direct access to a student's grades, thus, it will have to fetch students' details from the students microservice.
This microservice exposes three mappings:

- `POST /{studentId}` : insert a grade for a certain student. This route will check if the student exists using a HTTP request to the students microservice, as seen here;
- `GET /{studentId}` : gets all grades for a student given their id;
- `GET /` : gets all grades.

The schema is, again, really simple:

```sql
create table grades(
    id serial primary key,
    value int not null check(value > 0),
    student_id int not null
);
```

The aggregator

This application downloads the state of the databases of the two microservices mentioned above through HTTP calls and periodically (every 5 seconds) builds a school report for every student.
Nothing really exciting, you can check all the details here.

Checking that everything works

If you followed all the steps correctly, everything should work just fine. You can test your installation by using Postman to run some HTTP requests or by running the `check-base.sh` script at the root of this project.
You can also check the logs for the `aggregator` container to see if everything is working fine. Your output should be similar to this:

```
ReportCard{studentFullName='Aldo Tillo Poglis', studentGrades=[10, 9, 8, 4], avg = 7.75}
ReportCard{studentFullName='Whatever', studentGrades=[10, 9, 8, 4], avg = 7.75}
...
```

Now that everything is up and running, let's see how to implement authorization and authentication!

Next chapter: Chapter IV: implementing authentication
Previous chapter: Chapter II: the system and its components — how everything connects